07 — Autovacuum

> ***Module:*** *PostgreSQL Internals* | ***Difficulty:*** *Advanced* | ***Est. reading time:*** *50 min*

Table of Contents

Learning Objectives

After completing this section you will be able to:

- Describe the autovacuum launcher and worker architecture.
- Calculate the exact threshold at which autovacuum triggers for a given table.
- Explain per-table autovacuum storage parameters and when to use them.
- Monitor autovacuum effectiveness using `pg_stat_user_tables`.
- Identify tables that autovacuum cannot keep up with and apply appropriate tuning.
- Explain the relationship between autovacuum and XID wraparound prevention.
- Write bloat-detection queries to find tables needing maintenance.

Overview

Manual `VACUUM` is impractical at scale — you cannot run it manually after every batch of UPDATEs. **Autovacuum** is PostgreSQL's background maintenance daemon that automatically runs VACUUM and ANALYZE on tables when they cross configurable thresholds.

Autovacuum is **not optional** in production. Disabling it risks:

1. Unbounded table bloat (dead tuples accumulate).
2. XID wraparound (catastrophic data loss).
3. Stale query planner statistics (bad query plans).

The goal of autovacuum tuning is not to make it run less — it is to make it run effectively enough to prevent bloat without starving production queries.

Autovacuum Architecture: Launcher and Workers

```
postmaster
  ├── autovacuum launcher (1 process)
  │       Wakes up every autovacuum_naptime (1 min default)
  │       Scans pg_stat_user_tables for tables needing work
  │       Spawns worker processes (up to autovacuum_max_workers = 3)
  │
  ├── autovacuum worker 1  → vacuuming "orders"
  ├── autovacuum worker 2  → vacuuming "events"
  └── autovacuum worker 3  → analyzing "users"
```

Process	Role
Autovacuum launcher	Wakes up every `autovacuum_naptime`. Checks all databases; dispatches workers for tables needing vacuum/analyze.
Autovacuum worker	Does the actual VACUUM/ANALYZE work on a single table. Up to `autovacuum_max_workers` (default 3) can run simultaneously.

Each worker processes one table per invocation. If more tables need work than workers are available, the launcher queues them for the next naptime cycle.

```sql
-- See currently running autovacuum workers
SELECT pid, datname, query, state, wait_event_type, wait_event
FROM pg_stat_activity
WHERE query LIKE 'autovacuum:%';
```

When Autovacuum Fires: The Threshold Formula

Autovacuum vacuum triggers when:

```
n_dead_tup > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × reltuples
```

Where:

- `autovacuum_vacuum_threshold` (default **50**) — minimum dead tuples before autovacuum considers a table at all.
- `autovacuum_vacuum_scale_factor` (default **0.2**) — fraction of total live tuples that must be dead.
- `reltuples` — estimated total tuple count from `pg_class.reltuples`.

Example for a 1,000,000-row table:

```
threshold = 50 + 0.2 × 1,000,000 = 200,050 dead tuples

So autovacuum waits until 200,050 tuples are dead before vacuuming!
This is 20% of the table — potentially gigabytes of wasted space.
```

For a high-write table, tune aggressively:

```
-- For orders table (1M rows), trigger at 1% bloat instead of 20%:
ALTER TABLE orders SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 100
);
-- New threshold: 100 + 0.01 × 1,000,000 = 10,100 dead tuples
```

INSERT-triggered autovacuum (PG 13+)

From PostgreSQL 13, autovacuum can also trigger on INSERT-only tables to freeze tuples and prevent XID wraparound:

```
n_ins_since_vacuum > autovacuum_vacuum_insert_threshold +
                     autovacuum_vacuum_insert_scale_factor × reltuples
```

Defaults: threshold=1000, scale_factor=0.2.

Autovacuum ANALYZE Threshold

Autovacuum also triggers ANALYZE when:

```
n_mod_since_analyze > autovacuum_analyze_threshold + autovacuum_analyze_scale_factor × reltuples
```

Where:

- `autovacuum_analyze_threshold` (default **50**) — minimum modifications.
- `autovacuum_analyze_scale_factor` (default **0.1**) — fraction of total tuples.

ANALYZE keeps `pg_statistic` up to date for the query planner. Without fresh statistics, the planner may choose inefficient plans.

Cost-Based Delay for Autovacuum

Autovacuum workers use the same cost-based delay mechanism as manual VACUUM, but with their own parameters so they can be throttled independently:

Parameter	Default	Notes
`autovacuum_vacuum_cost_delay`	2 ms (PG 13+)	Sleep time per cost-limit cycle. 0 = no delay.
`autovacuum_vacuum_cost_limit`	-1	-1 means use `vacuum_cost_limit` (default 200).

The 2 ms default means autovacuum sleeps 2 ms for every 200 cost units consumed. This makes autovacuum about 5× slower than a full-speed VACUUM, preventing it from dominating I/O.

Problem: If the table is modified faster than autovacuum can process it at this throttled rate, dead tuples accumulate. Solutions:

1. Increase `autovacuum_vacuum_cost_limit` for that table.
2. Decrease `autovacuum_vacuum_cost_delay` for that table.

3. Increase `autovacuum_max_workers`.

Per-Table Storage Parameters

You can override global autovacuum settings for individual tables using storage parameters:

```
ALTER TABLE hot_table SET (
    autovacuum_enabled               = true,   -- can disable autovacuum on specific
table
    autovacuum_vacuum_threshold      = 100,
    autovacuum_vacuum_scale_factor   = 0.01,
    autovacuum_analyze_threshold     = 100,
    autovacuum_analyze_scale_factor  = 0.05,
    autovacuum_vacuum_cost_delay     = 0,       -- no delay for this table
    autovacuum_vacuum_cost_limit     = 800,     -- higher limit = vacuum goes faster
    autovacuum_freeze_min_age        = 5000000,
    autovacuum_freeze_max_age        = 100000000,
    autovacuum_multixact_freeze_max_age = 150000000,
    toast.autovacuum_vacuum_scale_factor = 0.01  -- also tune TOAST table
);

-- Verify per-table settings
SELECT reloptions FROM pg_class WHERE relname = 'hot_table';
```

Autovacuum and Freeze

Autovacuum handles freezing to prevent XID wraparound. It uses a separate, more aggressive threshold:

```
Aggressive (freeze) autovacuum fires when:
  age(relfrozenxid) > autovacuum_freeze_max_age (default 200,000,000 XIDs)

At that point, autovacuum performs a full table scan regardless of n_dead_tup,
replacing old XMINs with FrozenTransactionId.
```

This freeze autovacuum cannot be suppressed — PostgreSQL will force it even if `autovacuum = off`, because the alternative is eventual data loss from wraparound.

```
-- Tables closest to autovacuum freeze trigger
SELECT relname,
       age(relfrozenxid) AS xid_age,
       setting::int AS freeze_max_age,
       setting::int - age(relfrozenxid) AS xids_remaining
FROM pg_class
CROSS JOIN (SELECT setting FROM pg_settings WHERE name =
'autovacuum_freeze_max_age') s
WHERE relkind = 'r'
ORDER BY age(relfrozenxid) DESC
LIMIT 10;
```

Monitoring Autovacuum

Key columns in `pg_stat_user_tables`

Column	Meaning
`n_live_tup`	Estimated live tuple count
`n_dead_tup`	Estimated dead tuple count
`n_mod_since_analyze`	Modifications since last analyze
`n_ins_since_vacuum`	Inserts since last vacuum (PG 13+)
`last_vacuum`	Timestamp of last manual VACUUM
`last_autovacuum`	Timestamp of last autovacuum
`last_analyze`	Timestamp of last manual ANALYZE
`last_autoanalyze`	Timestamp of last autovacuum analyze
`vacuum_count`	Count of manual VACUUMs
`autovacuum_count`	Count of autovacuums

```
-- Full autovacuum health report
SELECT
    schemaname || '.' || relname         AS table,
    pg_size_pretty(pg_total_relation_size(relid)) AS size,
    n_live_tup,
    n_dead_tup,
    round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
    last_autovacuum,
    last_autoanalyze,
    autovacuum_count,
    now() - last_autovacuum              AS since_last_autovacuum
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC
LIMIT 20;

-- Tables autovacuum has not touched recently (> 24 hours)
SELECT schemaname || '.' || relname AS table,
       last_autovacuum,
       n_dead_tup
FROM pg_stat_user_tables
WHERE last_autovacuum < now() - interval '24 hours'
   OR last_autovacuum IS NULL
ORDER BY n_dead_tup DESC;

-- Autovacuum workers currently running
SELECT
    p.pid,
```

```
    p.datname,
    substring(p.query, 15, 40) AS table,
    age(clock_timestamp(), p.backend_start) AS running_for,
    pp.phase,
    pp.heap_blks_scanned * 100 / NULLIF(pp.heap_blks_total, 0) AS pct_done
FROM pg_stat_activity p
LEFT JOIN pg_stat_progress_vacuum pp ON pp.pid = p.pid
WHERE p.query LIKE 'autovacuum:%';
```

Bloat Detection Queries

```
-- Simple bloat estimate from pg_stat_user_tables
SELECT
    relname,
    n_live_tup,
    n_dead_tup,
    round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS bloat_pct,
    pg_size_pretty(pg_relation_size(relid)) AS table_size
FROM pg_stat_user_tables
WHERE n_dead_tup > 1000
ORDER BY n_dead_tup DESC;

-- More accurate bloat using pgstattuple (requires extension, does full scan)
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT
    relname,
    pg_size_pretty(table_len)           AS table_size,
    pg_size_pretty(dead_tuple_len)      AS dead_space,
    round(dead_tuple_percent::numeric, 2) AS dead_pct,
    pg_size_pretty(free_space)          AS free_space,
    round(free_percent::numeric, 2)     AS free_pct
FROM pgstattuple('orders')
CROSS JOIN (SELECT 'orders' AS relname) t;

-- Index bloat using pgstatindex
SELECT *
FROM pgstatindex('orders_pkey');
/*
  version | tree_level | index_size | root_block_no | internal_pages
| leaf_pages | empty_pages | deleted_pages | avg_leaf_density | leaf_fragmentation
*/

-- Comprehensive bloat report across all tables (expensive query)
SELECT
    schemaname, tablename,
    pg_size_pretty(heap_bytes) AS table_size,
    pg_size_pretty(bloat_bytes) AS bloat_size,
    round(bloat_ratio, 1) AS bloat_pct
FROM (
```

```
    SELECT
        schemaname, tablename,
        cc.relpages * bs::bigint AS heap_bytes,
        GREATEST(cc.relpages - CEIL((
            cc.reltuples * (
                nullhdr + ma - (CASE WHEN nullhdr % ma = 0 THEN ma ELSE nullhdr % ma
END)
                + datahdr
                + (1 - cc.reltuples / cc.relpages * is_na::int) * avg_width
            ) / (bs - page_hdr)
        )), 0) * bs AS bloat_bytes,
        GREATEST(round(
            100.0 * GREATEST(cc.relpages - CEIL((
                cc.reltuples * (...)  -- simplified
            )), 0) / cc.relpages, 1
        ), 0.0) AS bloat_ratio
    FROM pg_class cc
    JOIN pg_namespace nn ON nn.oid = cc.relnamespace
    -- ... complex formula from check_postgres or pgmetrics
    WHERE cc.relkind = 'r'
) sub
ORDER BY bloat_bytes DESC
LIMIT 20;
```

Tuning for High-Write Tables

The Problem

A table receiving 10,000 UPDATEs per second accumulates 864 million dead tuples per day. With default `scale_factor=0.2`, autovacuum only triggers at 20% bloat — by then the table may be 5–10× larger than necessary.

Solution: Aggressive per-table autovacuum settings

```
-- For a high-write OLTP table (e.g., orders, events, sessions)
ALTER TABLE orders SET (
    -- Trigger vacuum when 1% dead (not 20%)
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 100,

    -- Trigger analyze when 2% changed (not 10%)
    autovacuum_analyze_scale_factor = 0.02,
    autovacuum_analyze_threshold = 100,

    -- Remove cost delay so vacuum runs faster
    autovacuum_vacuum_cost_delay = 0,

    -- Higher cost limit allows more pages per cycle
    autovacuum_vacuum_cost_limit = 400
);
```

Increasing autovacuum parallelism

If 3 workers is not enough for your workload:

```
-- In postgresql.conf (requires restart)
-- autovacuum_max_workers = 6
-- autovacuum_naptime = 30s  (check more frequently)
```

Monitoring whether autovacuum is keeping up

```
-- Table is "behind" if n_dead_tup > threshold and last_autovacuum is stale
SELECT
    relname,
    n_dead_tup,
    last_autovacuum,
    now() - last_autovacuum AS lag,
    -- Calculate what the threshold is for this table
    (SELECT setting::float FROM pg_settings WHERE name =
'autovacuum_vacuum_threshold')
    + (SELECT setting::float FROM pg_settings WHERE name =
'autovacuum_vacuum_scale_factor')
    * reltuples AS trigger_threshold
FROM pg_stat_user_tables u
JOIN pg_class c ON c.oid = u.relid
WHERE n_dead_tup > 0
ORDER BY n_dead_tup DESC;
```

ASCII Diagrams

Autovacuum Decision Flow

```
autovacuum_naptime fires (every 60s default)
         │
         ▼
Launcher: scan pg_stat_user_tables for each database
         │
         ▼
For each table:
  n_dead_tup > threshold + scale_factor × reltuples?
         │
    YES  │              NO
         ▼              ▼
  Spawn worker      age(relfrozenxid) > freeze_max_age?
  → VACUUM               │
                    YES  │              NO
                         ▼              ▼
                 Spawn worker      n_mod_since_analyze > analyze_threshold?
                 → VACUUM FREEZE        │
                                   YES  │              NO
```

```
                                    ▼              ▼
                                Spawn worker      Skip table
                                → ANALYZE
```

Common Mistakes

1. **Setting `autovacuum = off`** for any database, even test. Risks wraparound and makes it easy to forget to turn back on.
2. **Not tuning `autovacuum_vacuum_scale_factor` for large tables.** For a 100-million-row table, the default 20% means 20 million dead tuples before vacuum fires — that's massive bloat.
3. **Raising `autovacuum_vacuum_cost_delay` to reduce I/O impact** without understanding the trade-off. Slowing down autovacuum means it cannot keep up with write rates.
4. **Ignoring `last_autoanalyze`.** A table can be vacuumed but not analyzed. Stale statistics cause bad query plans.
5. **Expecting `n_dead_tup` to be zero.** It is an estimate updated by the stats collector asynchronously. A small non-zero value is normal.
6. **Not monitoring TOAST autovacuum.** TOAST tables are separate relations; their autovacuum thresholds inherit from the main table's storage parameters, but can be set separately with `toast.*` prefix.

Best Practices

- **Never disable autovacuum.** At most, tune it more aggressively.
- **Lower `autovacuum_vacuum_scale_factor` to 0.01–0.05** for tables larger than 10 million rows.
- **Enable `log_autovacuum_min_duration = 250ms`** to log slow autovacuum runs for analysis.
- **Monitor `pg_stat_user_tables`** in your monitoring system: track `n_dead_tup` trends.
- **Consider `pg_repack`** for tables with severe bloat that cannot be compacted by regular vacuum.
- **Increase `autovacuum_max_workers` (to 5–10)** for large clusters with many high-write tables.
- **Set `autovacuum_naptime = 30s`** on busy OLTP systems to respond faster.

Performance Considerations

Parameter	Default	Production recommendation
`autovacuum_max_workers`	3	5–10 for busy clusters
`autovacuum_naptime`	1 min	30s for OLTP
`autovacuum_vacuum_scale_factor`	0.20	0.01–0.05 for large tables
`autovacuum_analyze_scale_factor`	0.10	0.02–0.05 for large tables
`autovacuum_vacuum_cost_delay`	2 ms	0–2 ms for tables under bloat pressure
`autovacuum_vacuum_cost_limit`	-1 (=200)	400–800 for high-write tables
`log_autovacuum_min_duration`	-1 (disabled)	250ms in production

Interview Questions

Q1. Describe the autovacuum launcher and worker architecture.

> *The launcher is a single background process that wakes up every `autovacuum_naptime` (default 1 minute), scans all databases for tables needing vacuum or analyze, and spawns worker processes. Up to `autovacuum_max_workers` (default 3) workers run simultaneously, each processing one table at a time.*

Q2. Write the formula for when autovacuum vacuum triggers.

> *`n_dead_tup > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor × reltuples`. With defaults: 50 + 0.2 × table_size. For a 1M-row table, this is 200,050 dead tuples.*

Q3. How would you tune autovacuum for a high-write table with 10 million rows?

> *`ALTER TABLE hot SET (autovacuum_vacuum_scale_factor = 0.01, autovacuum_vacuum_threshold = 100, autovacuum_vacuum_cost_delay = 0, autovacuum_vacuum_cost_limit = 400);` — triggers at 100,100 dead tuples (vs 2,000,050 with defaults) and runs without throttling.*

Q4. Why does autovacuum sometimes not keep up with a high-write workload?

> *The cost-based delay (`autovacuum_vacuum_cost_delay = 2ms`) throttles autovacuum, allowing other queries to use I/O. If the write rate generates dead tuples faster than throttled autovacuum can clean them, bloat grows. Solutions: reduce cost_delay, increase cost_limit, or add more workers.*

Q5. What column in `pg_stat_user_tables` tells you whether autovacuum is running often enough?

> *`last_autovacuum` (timestamp of last run) and `n_dead_tup` (current dead tuple count). If `n_dead_tup` is high and `last_autovacuum` is old, autovacuum is not keeping up.*

Q6. Can you turn off autovacuum for a single table?

> *Yes: `ALTER TABLE t SET (autovacuum_enabled = false)`. However, this is dangerous for tables that age significantly — the forced freeze autovacuum (anti-wraparound) will still fire regardless of this setting.*

Q7. What is `autovacuum_freeze_max_age` and why can't it be disabled?

> *It is the maximum XID age at which autovacuum will force a full-table freeze scan (default 200 million). It cannot be disabled because allowing a table to age beyond 2 billion XIDs would cause XID wraparound, making all tuples in that table invisible — effectively destroying the data.*

Q8. What does `log_autovacuum_min_duration` do?

> *When set (e.g., to 250 ms), it logs information about each autovacuum run that took longer than that threshold, including the table name, lock wait time, pages processed, tuples removed, and timing. Essential for diagnosing autovacuum performance issues.*

Exercises with Solutions

Exercise 1 — Calculate autovacuum trigger thresholds

```
-- Solution: Show current threshold for every user table
SELECT
```

```
    schemaname || '.' || relname AS table,
    n_live_tup AS live_rows,
    n_dead_tup AS dead_rows,
    -- Calculate current effective threshold
    (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_threshold')
    + (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_scale_factor')
    * reltuples AS vacuum_threshold,
    -- Is it over threshold right now?
    n_dead_tup > (
        (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_threshold')
        + (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_scale_factor')
        * reltuples
    ) AS needs_vacuum
FROM pg_stat_user_tables u
JOIN pg_class c ON c.oid = u.relid
ORDER BY (n_dead_tup / NULLIF(
    (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_threshold')
    + (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_scale_factor')
    * reltuples, 0
)) DESC NULLS LAST
LIMIT 20;
```

Exercise 2 — Set per-table autovacuum settings and verify

```
-- Solution
CREATE TABLE avac_test (id serial, data text);
INSERT INTO avac_test SELECT i, md5(i::text) FROM generate_series(1, 100000) i;

-- Check initial settings (none = uses global defaults)
SELECT reloptions FROM pg_class WHERE relname = 'avac_test';

-- Set aggressive settings
ALTER TABLE avac_test SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 50,
    autovacuum_analyze_scale_factor = 0.01,
    autovacuum_vacuum_cost_delay = 0
);

-- Verify
SELECT reloptions FROM pg_class WHERE relname = 'avac_test';

-- Reset to global defaults
ALTER TABLE avac_test RESET (
```

```
    autovacuum_vacuum_scale_factor,
    autovacuum_vacuum_threshold,
    autovacuum_analyze_scale_factor,
    autovacuum_vacuum_cost_delay
);
```

Exercise 3 — Autovacuum health dashboard

```
-- Solution: comprehensive autovacuum health query
WITH thresholds AS (
    SELECT
        (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_threshold') AS vac_thresh,
        (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_vacuum_scale_factor') AS vac_scale,
        (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_analyze_threshold') AS ana_thresh,
        (SELECT setting::numeric FROM pg_settings WHERE name =
'autovacuum_analyze_scale_factor') AS ana_scale
)
SELECT
    u.relname,
    pg_size_pretty(pg_total_relation_size(u.relid)) AS size,
    n_live_tup,
    n_dead_tup,
    round(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 1) AS dead_pct,
    t.vac_thresh + t.vac_scale * c.reltuples  AS vacuum_trigger,
    n_dead_tup > (t.vac_thresh + t.vac_scale * c.reltuples) AS needs_vacuum,
    n_mod_since_analyze,
    t.ana_thresh + t.ana_scale * c.reltuples  AS analyze_trigger,
    n_mod_since_analyze > (t.ana_thresh + t.ana_scale * c.reltuples) AS
needs_analyze,
    age(c.relfrozenxid)                       AS xid_age,
    last_autovacuum,
    last_autoanalyze
FROM pg_stat_user_tables u
JOIN pg_class c ON c.oid = u.relid
CROSS JOIN thresholds t
ORDER BY dead_pct DESC NULLS LAST
LIMIT 30;
```

Cross-References

- **06_vacuum.md** — VACUUM internals that autovacuum runs
- **08_visibility_map.md** — VM bits set by autovacuum
- **09_free_space_map.md** — FSM updated by autovacuum
- **10_system_catalogs.md** — `pg_stat_user_tables` in detail